

Perceptrón Simple con Datos Generados Aleatoriamente

¿Qué hace este código?



GENERA UN CONJUNTO DE
PUNTOS ALEATORIOS
BIDIMENSIONALES.



TRAZA UNA LÍNEA DE SEPARACIÓN
ARBITRARIA $Y = 2X$.



CLASIFICA LOS PUNTOS CON BASE
EN ESTA LÍNEA.



APLICA EL ALGORITMO DE
PERCEPTRÓN PARA ENCONTRAR
UNA FRONTERA DE DECISIÓN.

Generación de puntos aleatorios

- Se generan 20 puntos dentro del rango (-1,1). Las coordenadas se separan en dos vectores: x1 y y1. Se grafican inicialmente con asteriscos.

```
15 nct=20; //tamaño del conjunto de trabajo
16 x=2*rand(2,nct)-1;
17 x1=x(1,:); //Arreglo x1 contiene coordenadas x
18 y1=x(2,:); //Arreglo y1 contiene coordenadas y
19 plot(x1,y1,'*');
20
```

Definición de la recta de referencia

- Se define la línea $y = 2x$ como referencia.
- Se representa con el vector de coeficientes $F = [1; -2]$.
- Se grafica en rojo sobre el plano.

```
23 F=[1;-2];
24 //la función hipotesis es g(x,y)=w1*x+w2*y
25 //pesos iniciales son w1=0 and w2=0
26 w=[0;0];
27
28 //mostramos área de trabajo para fines visuales solamente
29 x2=linspace(-1,1,100);
30 for i=1:100
31     y2(i)=2*x2(i);
32 end
33 plot(x2,y2,'r') //trazamos una línea roja
```

Clasificación previa de los puntos

- Se calcula la diferencia entre la posición del punto y la línea.
- Se clasifica cada punto usando la función `sign()`.
- Se grafican en verde o azul según su clase.

```
35 //Clasificamos los puntos a la derecha e izquierda
    de la linea
36 //los puntos tienen la misma x, solo hay que calcul
    ar la y del punto
37 //y la y de la recta y restar,
38 //clasificamos segun el resultado
39 for i=1:nct
40     l(i)=-F(2)*x1(i)-y1(i); //l(i)=F(2)*y1(i)+F(1)*x
        l(i);
41     class_F(i)=sign(l(i));
42 end
43
44 for i=1:nct
45     if class_F(i)==1 then
46         plot(x1(i),y1(i),'gre*');
47     else
48         plot(x1(i),y1(i),'blu*');
49     end
50 end
51 ///// fin de generacion de conjunto de trabajo ///
```

Aplicación del algoritmo de perceptrón

```
57 for i=1:nct
58     g(i)=sign(w(2)*y1(i)+w(1)*x1(i));
59 end
60 //~= operador relacion1 es distinto que
61 //buscamos las salidas que no concuerdan con nuestra
62 //clasificación
63 ind = find(g ~= class_F);
```

- Se inicializan los pesos $w = [0; 0]$.
- Se calcula la salida del perceptrón con los pesos actuales.
- Se comparan los resultados con la clasificación original.

Actualización de pesos

- Mientras existan errores, se actualizan los pesos.
- Se usa la diferencia entre la clase esperada y la calculada.
- El ciclo termina cuando todos los puntos están correctamente clasificados.

scilab

```
65 while ~isempty(ind) //si es vacio => fin
66     //actualizamos los pesos w
67     //for i=1:nct
68     //operador .* es multiplicación de matrices el
    elemento a elemento
69     w(1) = w(1) + (class_F(ind(1))-g(ind(1))).*x1(
ind(1)); //
70     w(2) = w(2) + (class_F(ind(1))-g(ind(1))).*y1(
ind(1)); //
71     //end
72     //temp = sign(w'*x);
73     for i=1:nct
74         temp(i) = sign(w(2)*y1(i)+w(1)*x1(i));
75     end
76     ind = find(temp ~= class_F);
77     iter = iter + 1;
78     g = temp;
79 end
```

Resultado del entrenamiento

- Se traza la nueva línea de decisión aprendida por el perceptrón.
- Esta línea es distinta de la original, pero separa correctamente los puntos.

```
80 for i=1:100
81     y3(i)=(-w(1)*x2(i))/w(2);
82     //y3(i)=-(w(2)/w(1))*x2(i);
83 end
84 plot(x2,y3,'g');
```


Visualización final

- En verde: línea aprendida por el perceptrón.
- En rojo: línea original de referencia.
- Puntos azules y verdes según su clase.

